

Project Documentation

CI/CD Pipeline for Java Application with Jenkins

1. Project Objective

- To build a fully automated CI/CD pipeline using Jenkins on AWS to compile, test, and deploy a Java web application to a Tomcat server.

2. Problem Solved

- The previous manual deployment process was slow, error-prone, and delayed feature releases.
- This project automated the entire workflow to make deployments fast, reliable, and consistent.

3. My Role & Responsibilities (Solo Project)

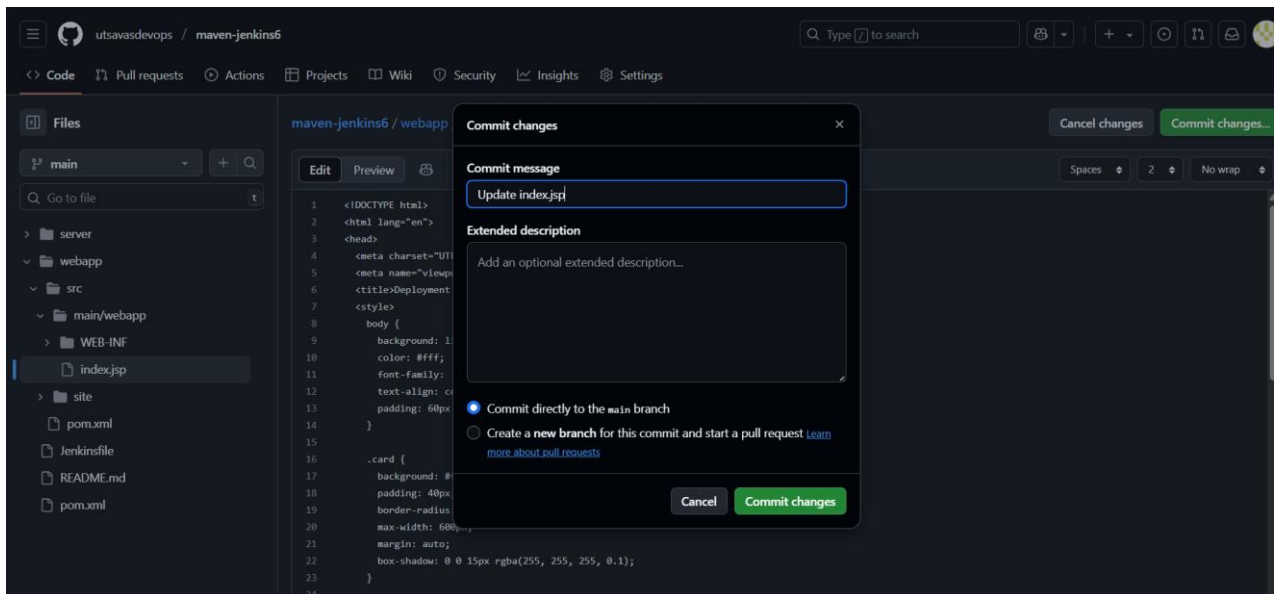
- **Cloud Setup:** Deployed and configured an AWS EC2 instance to host the Jenkins server.
- **Jenkins Configuration:** Installed Jenkins, Java, Tomcat, and managed key plugins (Copy Artifact, Pipeline Stage View, Deploy to Container).
- **Pipeline Design:** Architected a two-job pipeline (build and deploy) to separate concerns.
- **Automation Scripting:** Wrote and configured the Groovy pipeline scripts directly in the Jenkins UI for both the build and deploy jobs.
- **Integration:** Connected Jenkins to GitHub using webhooks for automatic pipeline triggers on code commits.
- **Troubleshooting:** Identified and fixed all pipeline-related issues.

4. Technology Stack

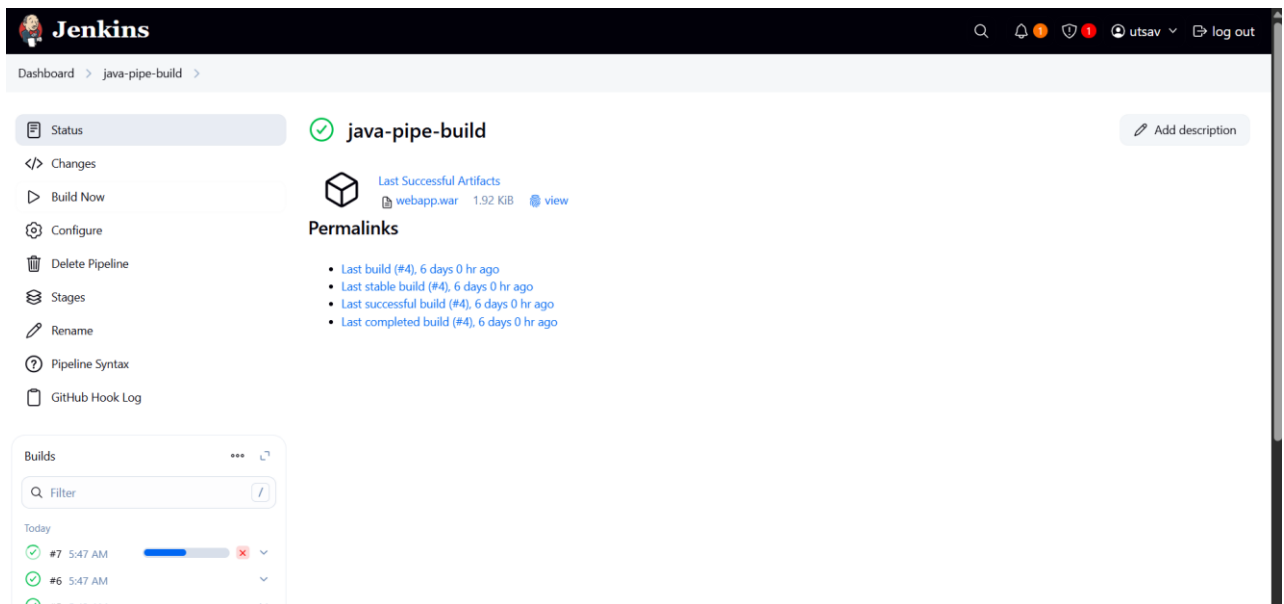
- **Cloud:** AWS EC2
- **CI/CD:** Jenkins
- **Language:** Java
- **Build Tool:** Maven
- **Version Control:** Git & GitHub (Webhooks)
- **Application Server:** Apache Tomcat 9
- **Scripting:** Groovy, Shell

5. Automated Workflow Steps

- 1. **Code Push:** I push new code to the GitHub repository.



- 2. **Auto-Trigger Build:** A GitHub webhook instantly triggers the java-pipe-build job in Jenkins.

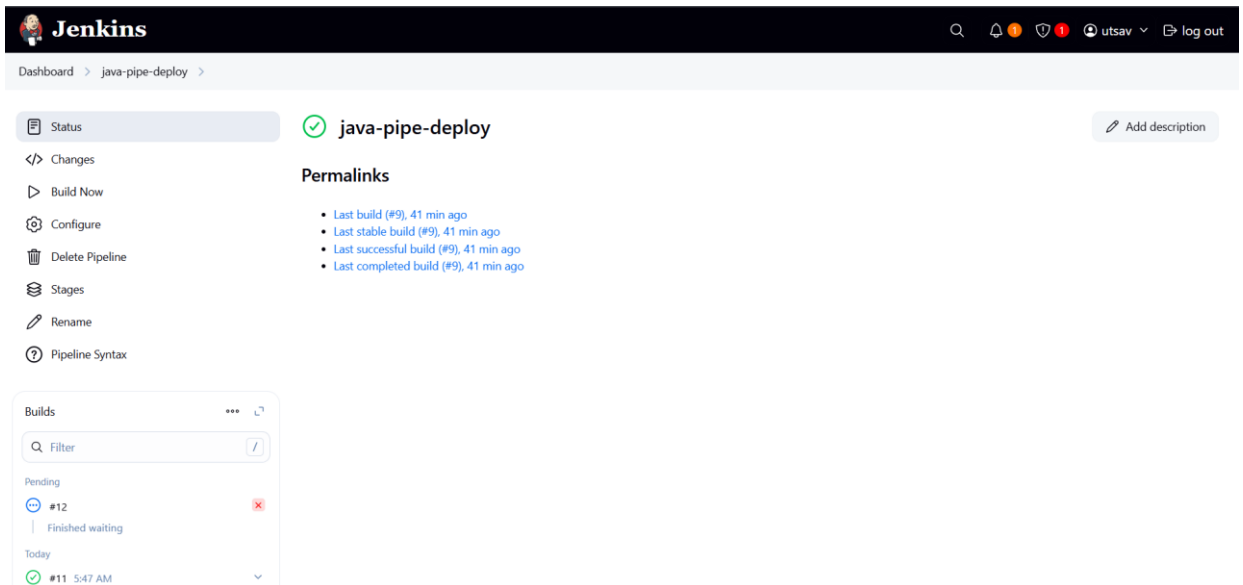


3. Build & Archive: The job downloads the code, builds it with Maven, and archives the .war file.

```
Dashboard > java-pipe-build > #7

[Pipeline] {
[Pipeline] archiveArtifacts
Archiving artifacts
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (BUILD other project)
[Pipeline] tool
[Pipeline] envVarsForTool
[Pipeline] tool
[Pipeline] envVarsForTool
[Pipeline] withEnv
[Pipeline] {
[Pipeline] build (Scheduling java-pipe-deploy)
Scheduling project: java-pipe-deploy
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

- **4. Trigger Deploy:** The build job automatically starts the java-pipe-deploy job.



The screenshot shows the Jenkins web interface for the 'java-pipe-deploy' job. The top navigation bar includes the Jenkins logo, search, notifications, and user profile (utsav). The breadcrumb trail is 'Dashboard > java-pipe-deploy'. On the left sidebar, the 'Status' tab is selected, showing options like 'Changes', 'Build Now', 'Configure', 'Delete Pipeline', 'Stages', 'Rename', and 'Pipeline Syntax'. The main content area displays the job name 'java-pipe-deploy' with a green checkmark icon and an 'Add description' button. Below this, a 'Permalinks' section lists build history: 'Last build (#9), 41 min ago', 'Last stable build (#9), 41 min ago', 'Last successful build (#9), 41 min ago', and 'Last completed build (#9), 41 min ago'. At the bottom, a 'Builds' table shows a pending build #12 and a completed build #11 at 5:47 AM.

- **5. Copy & Deploy:** The deploy job copies the `.war` artifact and deploys it to the Tomcat server.

Dashboard > java-pipe-deploy > #12

Status

Changes

Console Output

Edit Build Information

Delete build '#12'

Timings

See Fingerprints

Pipeline Overview

Restart from Stage

Replay

Pipeline Steps

Workspaces

Previous Build

Console Output

Download Copy View as plain text

```

Started by upstream project "java-pipe-build" build number 7
originally caused by:
  Started by user utsav
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/lib/jenkins/workspace/java-pipe-deploy
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Copy Artifact)
[Pipeline] copyArtifacts
Copied 1 artifact from "java-pipe-build" build number 7
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Deploy war/ear to container)
[Pipeline] deploy
[DeployPublisher][INFO] Attempting to deploy 1 war file(s)
[DeployPublisher][INFO] Deploying /var/lib/jenkins/workspace/java-pipe-deploy/webapp/target/webapp.war to container Tomcat 9.x Remote with context null
  Redeploying [/var/lib/jenkins/workspace/java-pipe-deploy/webapp/target/webapp.war]
  Undeploying [/var/lib/jenkins/workspace/java-pipe-deploy/webapp/target/webapp.war]
  Deploying [/var/lib/jenkins/workspace/java-pipe-deploy/webapp/target/webapp.war]
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS

```

- **6. Verification:** The updated application is live and accessible via its URL.

← → ↻ ⚠ Not secure 65.2.81.249:8090/webapp/ ☆ ⓘ

âœ...

Deployment Successful!

Your Java web application has been successfully deployed using Jenkins CI/CD Pipeline.

Visit My GitHub

© 2025 Utsav Prajapati | DevOps Project Demo

6. Core Pipeline Scripts (Configured in Jenkins UI)

- **Build Job (java-pipe-build):** Downloads, compiles, and archives the application.

```
pipeline {
    agent any
    tools {
        maven 'maven'
        jdk 'java17'
    }
    stages {
        stage('Download code from GIT') {
            steps {
                git branch: 'main', url:
'https://github.com/utsavasdevops/maven-jenkins6.git'
            }
        }
        stage('Build') {
            steps {
                sh 'mvn clean package'
            }
        }
        stage('Archive Artifact') {
            steps {
                archiveArtifacts artifacts: '**/*.war', followSymlinks:
false
            }
        }
        stage('BUILD other project') {
            steps {
                build wait: false, job: 'java-pipe-deploy'
            }
        }
    }
}
```

- **Deploy Job (java-pipe-deploy):** Copies the artifact and deploys it to the container.

```
pipeline {
    agent any
    stages {
        stage('Copy Artifact') {
            steps {
                copyArtifacts filter: '**/*.war', fingerprintArtifacts:
true, projectName: 'java-pipe-build', selector: lastSuccessful()
            }
        }
        stage('Deploy war/ear to container') {
            steps {
                deploy adapters: [tomcat9(credentialsId: '4999bf5e-7d4e-
4456-9ae3-f0fc9376d94c', path: '', url: 'http://65.2.81.249:8090/)],
contextPath: null, war: '**/*.war'
            }
        }
    }
}
```

7. Challenges & Solutions

- **Challenge:** A single pipeline for build and deploy would be complex and hard to debug.
- **Solution:** I created a modular, two-job pipeline. This separation makes the process cleaner, easier to manage, and allows for re-running only the deployment if necessary.

8. Project Outcomes

- **Result:** A fully functional, hands-free CI/CD pipeline.
- **Impact:** Reduced deployment time from hours to minutes and eliminated manual errors.
- **Key Learning:** Gained practical experience in DevOps automation, Jenkins, AWS, and pipeline scripting.